

SkillClaw

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

LLM agent skills continuously evolve by aggregating cross-user interaction trajectories and applying agentic, evidence-driven updates.

arxiv.org/abs/2604.08377

ML researchers + AI systems engineers

Closed-loop premise

Runtime interactions become shared evidence; verified mutations flow back into the skill ecosystem.

Agent Skills Remain Static After Deployment

Today: static skill libraries

Users manually install skills from a centralized hub. Skills rarely persist learning from runtime interaction beyond individual sessions.

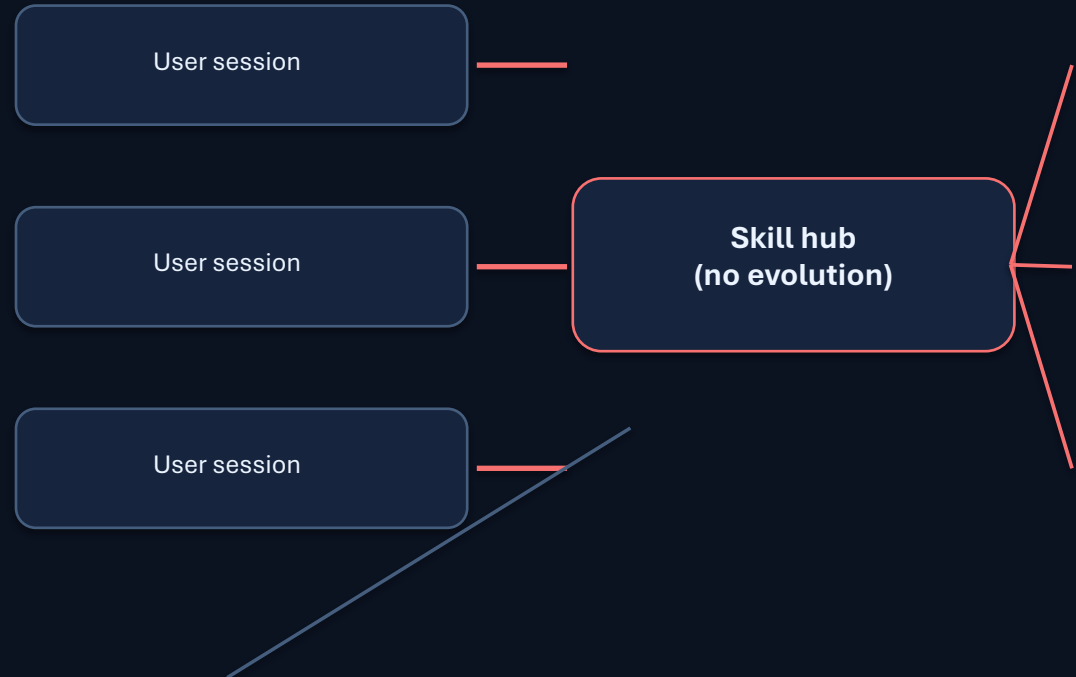
Repeated rediscovery

As similar tasks recur across users and time, the same failure and recovery patterns reappear without system-level accumulation.

Why existing methods fall short

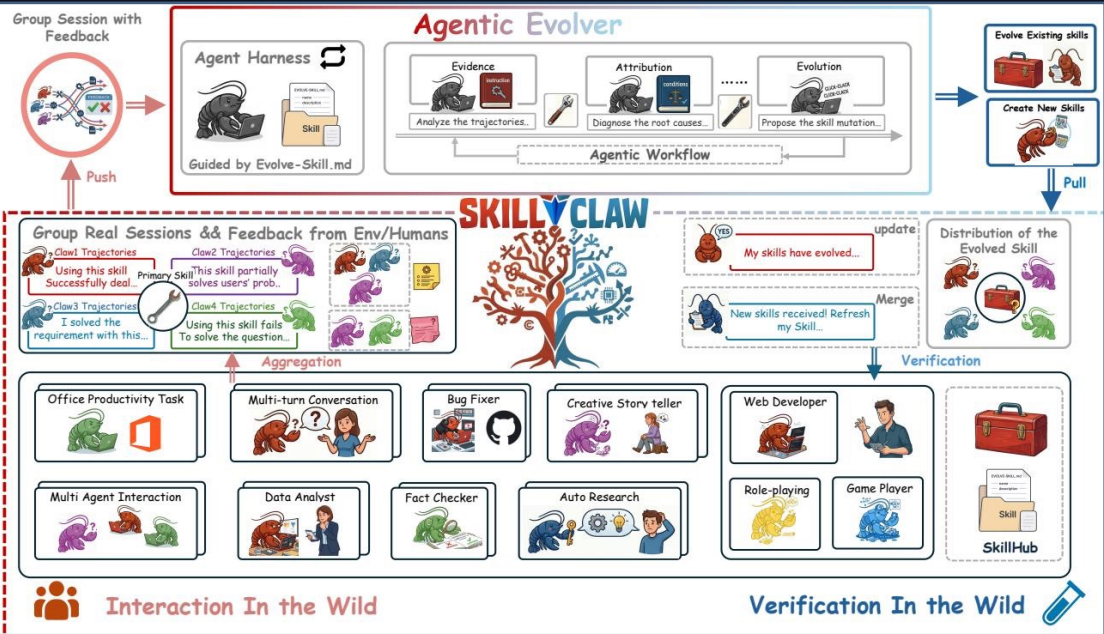
Memory methods stay instance-tied; skill methods compress experience into instructions but treat the library as static.

Static paradigm

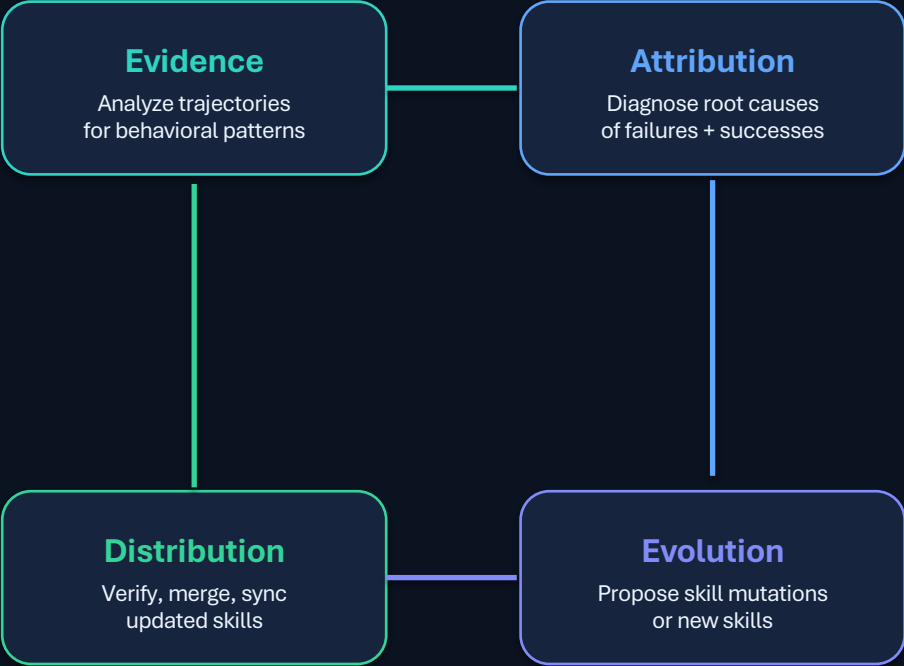


Same solution rediscovered independently

Closed-Loop Pipeline: Multi-User Interaction → Shared Skill Evolution



Source architecture figure: multi-user interaction, session collection, evolver, synchronization.



Multi-user interaction

Shared skill repository

Agentic Evolver: Evidence → Attribution → Evolution

1

Evidence

Structured session trajectories preserve full action–feedback causal chains. The evolver identifies behavioral patterns across grouped sessions.

2

Attribution

Root causes of failures and successes are diagnosed from the evidence instead of using predefined update rules.

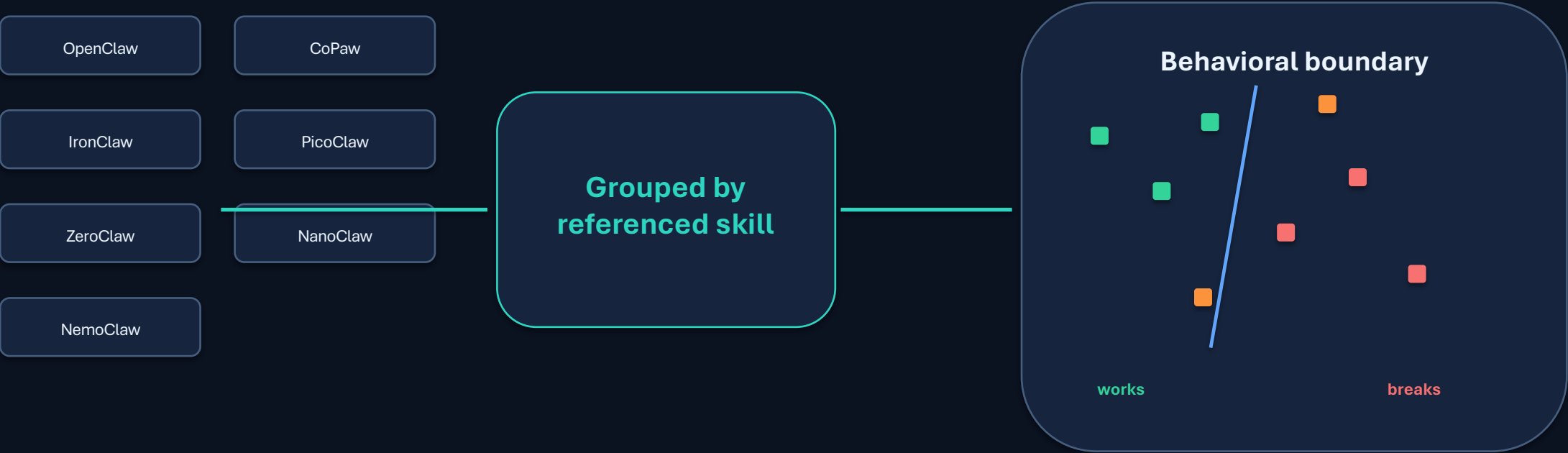
3

Evolution

The agent proposes mutations: refining existing skills or creating new ones, then candidate skills enter verification.

Core distinction: autonomous open-ended reasoning over interaction evidence — not a hand-written rule engine.

Cross-User Trajectory Aggregation Exposes Behavioral Boundaries



Why aggregation matters

Different users exercising the same skill under diverse contexts produce complementary views: success patterns and failure modes.

Single-user limitation

One user rarely produces enough signal to separate a generalizable improvement from an idiosyncratic fix.

Case Study: Multimodal Creative Pipeline Skill — 6 Days

Outcome: only 1 of 6 candidate skills was accepted, illustrating regression-preventing verification.

Day	Date	Outcome	Rationale
1	Day 1	Accept	validate-tmp-workspace-inputs checked /tmp_workspace inputs and environment setup; upgraded to new best pool.
2	Day 2	Reject	multimodal-input-validation-and-setup proposed validation and output env init; current best pool kept.
3	Day 3	Reject	multimodal-creative-task-pipeline extracted from PDF/video/image; current best pool kept.
4	Day 4	Reject	Improved pipeline added image classification, visual generation, structured validation; current best pool kept.
5	Day 5	Reject	Creative pipeline plus per-file fail-fast validation via validate-required-input-files; current best pool kept.
6	Day 6	Reject	Candidate extended PDF-to-poster and document-to-visual paths; not admitted to next cycle.

Case Study: Git Push Auth-Fallback Skill — Iterative Refinement

Outcome: 3 of 6 days yielded accepted updates; later attempts plateaued under validation.

Day	Date	Outcome	Rationale
1	Day 1	Accept	git-push-with-auth-fallback introduced safe fallback instead of blocking on push failure; added to Safety best pool.
2	Day 2	Accept	Unified patch/bundle filenames and reduced inconsistency; updated best pool kept.
3	Day 3	Accept	Added auth-alternative paths and secrets audit with git-clone-to-directory; fixed subshell pitfalls.
4	Day 4	No update	Same-pool retest confirmed current pool as best; no new candidate admitted.
5	Day 5	Reject	Added “push hang treated as auth failure,” but validator found no improvement; current pool kept.
6	Day 6	Reject	Added identity config and filename consistency; no improvement and not admitted to next cycle.

Note: Day 4 is a same-pool retest accepted by the validator, but no new candidate update was admitted.

Three Properties Distinguish SkillClaw from Static Skill Systems

1 Collective evolution

Individual interactions improve a shared, continuously improving skill ecosystem.

2 Fully automatic

Runtime interaction drives evolution without manual curation or explicit user intervention.

3 Agentic evolution paradigm

Skill updates arise from open-ended reasoning over evidence rather than predefined update rules.

Implication for agent systems

Treat deployed skills as living infrastructure: collect trajectories, attribute behavior, verify mutations, and distribute improvements back to compatible agents.

- Static ecosystems force rediscovery; collective evidence enables accumulation.
- Conservative validation can reject most candidates while preserving improvement paths.
- Multi-user diversity reveals when a fix is generalizable rather than idiosyncratic.